

Document 19 — MLOps & LLMOps (Running What You Built)

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Added at the end on purpose: MLOps only makes sense once there's a real system to run — you need Project 5 (Docs 12-18) working before "how do I safely change it in production" is a real question, instead of just a theoretical one.

Prerequisites

[18_capstone](#) (Project 5 built and working)

How to Read & Practice This Document

- **What:** the discipline of safely changing, deploying, and watching an AI system *after* it's already live — versions, gated releases, rolling back, catching slow decline.
- **Why:** Doc12 taught you how to ship a system once. This document is about what happens on day 2 — when you want to change a prompt, swap a model, or fix an agent, without breaking what's already working for real users.
- **When:** the moment your system has real users (or you're treating it as if it will) — not important for a one-off script, essential for anything you'd call "production."
- **How to practice:** 1. Read the Core Concepts below — this document's outside reading is kept small on purpose, since MLOps for LLM systems is a newer, faster-changing area than the rest of this curriculum, without as much settled reference material. 2. Do the **Basic/Intermediate** exercises against your own Project 5. 3. Do the **Real-world/Failure** exercises for real — actually trigger a rollback, actually act out a slow-decline situation. 4. Use **Hint 1** → **Hint 2** only after really trying. Ask for the full solution only if you say "**Show me the solution.**" 5. Before calling this "done," explain out loud what would need to be true for you to trust an automatic rollback enough to not watch every release yourself. If you can't, you're not done.

The Story — what this document is actually building

Picture Project 5 already live, with real people depending on it, and you want to improve one of its prompts. Everything in this document is one connected story about how to make that change safely

— the same discipline a classic ML team already applies to a newly retrained model, just aimed at prompts and settings instead, since here nothing gets retrained.

- **First, the prompt has to become a real, tracked thing.** Not text you edit in place and hope you remember what changed — a named, saved version, so "which version was live when this happened" is always an answerable question.
- **Second, a new version has to earn its way to production.** Doc13 already built a test suite that can catch a regression when you run it by hand — the release gate is what makes running it *automatic and mandatory*, not optional discipline that's easy to skip under time pressure. No passing score, no release.
- **Third, even a version that passed the gate doesn't go straight to everyone.** A canary release hands it a small slice of real traffic first, or a shadow release runs it silently alongside the old version with nobody seeing its answers, so a problem the test suite missed shows up in real numbers before it reaches your whole user base.
- **Fourth, "healthy at launch" isn't the end of the story.** Costs creep up, the questions users ask drift, the documents behind a RAG system go stale — a slow decline like that never throws an error, so the same kind of score that gated the release has to keep being tracked over time, watching for a trend, not just a single bad request.
- **Fifth, the moment the gate or the ongoing watching shows a version is actually worse, rollback is the safety net.** It snaps the "live" pointer back to the last version *known to have passed* — automatically, not "whatever came before, good or bad," and not waiting on a person to notice a dashboard.

All five pieces answer one question: how do you keep changing an AI system without ever betting your whole user base on an untested guess? That's exactly what this document's Build Task turns into one real, working gate for Project 5. It's also why proving the gate can't be quietly bypassed, and having a backup plan for when your one outside provider has an outage, both matter just as much — a gate someone can walk around, or a system with no plan B for its only dependency, isn't actually safety. It just looks like it, right up until the day it isn't.

Jump to: [Core Concepts](#) · [Practice Exercises](#) · [Build Task](#)

Core Concepts (read this first — everything you need is here)

MLOps vs. LLMOps: what's actually different here

Classic MLOps centers around a training loop: retrain a model on new data, check it, ship it, watch for the data changing in ways that would mean it's time to retrain again. Most of what you build with LLMs skips that loop completely — you're not training the model, you're changing *prompts, tool settings, search sources, and the logic that connects them all* around a model you don't control. **Why this difference matters:** it means "LLMOps" borrows MLOps's discipline (versions, gated releases, watching, rolling back) but applies it to a different set of things — a prompt change is your version of

"a new model," and it deserves exactly the same care a classic ML team would give a newly retrained model, even though nothing was actually trained.

Versions for prompts/settings: treating them as real, tracked things

A prompt that shapes an agent's behavior isn't "just some text" — it has as much power over behavior as code does, and it needs the same care: kept in version control (in git, like code), changed through reviewable diffs, and tagged so you can tell exactly which version was live for any past run. **Why this matters in practice:** when quality gets worse in production, "which prompt version was live when this bad answer happened" needs to be a question you can actually answer. Without keeping versions, it isn't, and fixing a regression turns into guesswork.

CI/CD for agent systems: the test suite as a release gate

Doc13 built a test suite that can catch a regression when you run it by hand. **LLMOps makes that automatic:** no prompt or model-setting change should reach production without the test suite running against it first, with a clear score line that blocks the release if it's not met — the exact same idea as tests blocking a bad code merge in normal software, just measuring a different kind of correctness. **Why this is worth automating instead of just trusting people to remember:** the whole point of Doc13's regression-catching suite is defeated if it's optional and easy to skip under time pressure — which is exactly when a bad change is most likely to slip through.

Ways to release: canary, blue-green, and shadow

Canary release — a new version handles a small share of real traffic (say 5%) while the old version keeps handling the rest; if the canary's numbers (errors, test scores, cost, speed) look healthy after a set time, it gradually takes over the rest of the traffic. **Blue-green** — two full copies of the system exist (old = "blue," new = "green"); traffic switches over all at once, but the old copy stays ready, so rolling back just means switching traffic back, not a slow re-release. **Shadow release** — the new version runs on real traffic *at the same time* as the old one, but its output is only recorded and compared, never actually shown to users — the safest way to test a risky change on real inputs, with zero risk to users. **Why these matter more for agent systems than typical web apps:** an agent's failure is often quiet — a slightly worse but still believable-sounding answer — which canary/shadow methods are made specifically to catch, by comparing numbers, before every user sees it.

Watching for slow decline: cost, speed, and quality over time

Slow decline, in this sense, isn't about the model changing underneath you (though a provider quietly updating a model version is a real risk worth watching for) — it's about your system's *inputs* changing over time in ways your prompts/tools weren't built for: how users behave shifts, the documents in your RAG set go out of date, tool APIs change what they send back. **Why this needs active watching, instead of just "it'll show up eventually":** these are exactly the failures that don't throw an error — rising cost, growing delay, or test-style quality scores slowly dropping are all

trends you only catch by tracking numbers over time, not by any single request looking obviously broken.

Rolling back: the real safety net

Rolling back means going back to the last version you know was good — a prompt, a model choice, or a whole release — the moment watching or a gated test shows the new version is worse. **Why an automatic, already-defined rollback trigger matters more than a person "watching a dashboard"**: people get busy, alerts get missed, and the whole value of rolling back is in how fast it happens after a problem starts — a rollback that needs a person to notice, decide, and manually run it is much slower (and much less reliable) than one that fires automatically when a number crosses a set line.

Backup plans if your provider has an outage

Your system depends on an outside provider (OpenAI) that can have outages, rate-limit changes, or model retirements, all outside your control. **Why this is a real risk, not just a hypothetical**: a production agent system with no backup plan goes fully down the moment its one dependency does. **What a basic backup plan looks like**: a second model/provider set up to take over after repeated failures, and a graceful fallback (a clear "temporarily unavailable" message) as the last resort — better than the raw error a user would otherwise see.

Practice Exercises

Setup for this document's practice code: work inside `19_mlops_llmops/` (same venv as before — if it's not active, `source .venv/bin/activate`). New package for this document: `pip install pytest`.

How to run each exercise: save it as its own small script — `practice_basic.py`, `practice_intermediate.py`, and so on, matching the levels below — and run it directly: `python practice_basic.py`. Keep each one runnable on its own; don't chain them into one file.

Jump to an exercise: [Basic](#) · [Intermediate](#) · [Real-world](#) · [Edge cases](#) · [Failure](#) · [Build Task](#)

Basic — a simple version log

- **What**: tag two versions of one prompt from Project 5 (v1, v2) in a simple version log, and run Doc13's test suite against both.
- **Why**: you can't have a deploy gate without something to compare against first — this is the record that makes "which version was live when" an answerable question.
- **When you'll hit this for real**: the very first time you change a prompt in a system you actually care about not breaking.

- **How to code it:** a plain JSON or SQLite record with `{version_name, prompt_text, created_at}` , saved for two variants of one prompt, with your Doc13 suite run against each and the scores printed side by side.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate — build the gate

- **What:** a small release gate script that refuses to update the "current live prompt version" unless the new version's test score meets or beats a set bar.
- **Why:** this is the actual mechanism that turns "we should test before releasing" from a policy people can forget into something the system enforces.
- **When you'll hit this for real:** this document's own Build Task, protecting Project 5.
- **How to code it:** a `deploy(version_name, threshold)` function that runs your test suite, and only updates a `live_version.txt` (or database row) if the score clears the threshold — otherwise it prints why and exits without updating anything.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Real-world — act out a canary release

- **What:** send a small share of test requests to v2, the rest to v1, and compare live numbers (cost, speed, test score) between the two groups.
- **Why:** this is the actual pattern real companies use to de-risk a change — practicing it here, on your own project, is what makes it something you can describe concretely in an interview.
- **When you'll hit this for real:** rolling out any real prompt or model change to a system already serving real traffic.
- **How to code it:** loop over your test requests, route roughly 20% to v2 and the rest to v1 (`random.random() < 0.2`), log which version handled each one, and compare aggregate scores/cost between the two groups at the end.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Edge cases — a bad release that slips past the gate

- **What:** deliberately release a worse version past your own gate (skip it on purpose), and confirm your watching would have caught it — then wire the gate back so it can't be skipped quietly.
- **Why:** a gate that can be bypassed by editing a file directly isn't really a gate — you need to prove to yourself it can't happen by accident.
- **When you'll hit this for real:** exactly the failure mode this document's Break-It section warns about — someone (possibly future you) editing the "live" pointer directly under time pressure.
- **How to code it:** manually edit your `live_version.txt` to point at an untested version, bypassing `deploy()` . Confirm nothing in your system stops you — then add a check (a hash, or a "passed_gate: true" flag) that makes a manually-edited pointer detectable or impossible.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Failure — a real rollback trigger

- **What:** define a real rollback trigger (like "test score drops more than X% over Y requests") and act out the situation that should set it off.
- **Why:** a rollback plan that only exists as an idea, never tested, is not a rollback plan — you need to have watched the trigger actually fire once.
- **When you'll hit this for real:** the day a released version turns out to be worse than it looked in testing — this is what makes recovery fast instead of panicked.
- **How to code it:** write a function checking recent scores against a threshold, feed it a simulated sequence of scores that crosses the line partway through, and confirm it correctly calls your `rollback_to_previous()` function at the right moment.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Build Task — Versioned Release Gate for Project 5

Stuck on the Build Task? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#) **Goal:** a light versioning and gated-release layer on top of Project 5, so changing a prompt or agent setting in production requires passing the same test suite from Doc13 — not just optional, easy-to-skip discipline.

Requirements:

- Prompts/settings for Project 5's agents are stored as versioned, named pieces (not edited in place, with no history).
- A `deploy(version)` step that runs the Doc13 test suite against that version, and refuses to mark it "live" if the score is below a set bar.
- A record of which version was live at any point in time, so a given past run can be traced back to the exact prompt/setting that produced it.
- A rollback function that puts the "live" marker back to the last version that actually passed.

Inputs: a new prompt/setting version to release; the existing test suite from `13_testing_evaluation_observability`.

Outputs: either the version goes live (it passed the gate), or the release is refused, with the test scores explaining why.

Constraints: the gate must not be skippable by accident — "just editing the prompt file directly" shouldn't be how a version reaches production.

Suggested files:

```
19_mlops_llmops/
├─ versioning.py
├─ deploy_gate.py
```

```
|— rollback.py
|— test_deploy_gate.py
```

Functions/Components to build:

- `versioning.py` → `save_version(name, config) -> VersionRecord` ,
`get_version(name) -> VersionRecord`
- `deploy_gate.py` → `deploy(version_name: str, threshold: float) -> DeployResult`
(runs Doc13's test suite inside itself)
- `rollback.py` → `rollback_to_previous() -> VersionRecord`

Expected Behavior

- A version that scores above the bar releases, and becomes traceable as "live."
- A version that scores below the bar is refused, with the specific failing test cases shown.
- Rolling back correctly restores the last version that passed, not just "whatever came before" no matter if it passed or not.

Test Cases

Scenario	Expected
New version scores above the bar	Releases, becomes live, saved with a version ID
New version scores below the bar	Release refused, failing test cases shown
Rollback called after a bad release got through	Live marker goes back to the last version known to be good
Asking "what was live at time T"	Returns the correct saved version record

Break-It / Debug Preview

- A release gate that can be skipped by editing the "live" marker directly — completely defeating its purpose.
- A rollback that goes back to "whatever came before" instead of "the last one that actually passed" — quietly re-releasing a version that was already broken.
- A slow decline that only shows up over weeks (rising cost) instead of one bad request — nothing catches it without deliberately tracking the trend, not just per-request alerts.

Interview Topics Preview

- MLOps vs. LLMOps · why prompts need the same version discipline as code · canary vs. blue-green vs. shadow release trade-offs · what a good rollback trigger looks like · a backup plan for a provider outage.

□ You Can Now Finish Project 5

Docs 12-19 are everything Project 5 needs — this document is the last piece (the versioned deploy gate). Go to [project_5_production_capstone/](#) and follow its Step 4 to add this document's gate on top of what Docs 12-18 already built.

Move On When

You can explain, clearly, what would need to be true for you to trust an automatic rollback enough not to watch every release yourself — and you have a working gate that actually blocks a worse version of Project 5 from reaching "live." There's no document after this one — go back to [18_capstone](#) and [CURRICULUM.md §6](#) to confirm the whole curriculum is complete.

Stuck? Ask for **Hint 1** or **Hint 2**. Ask for the full solution only if you say "**Show me the solution.**"